AIM 2627 C++ Assessment 01

Sentry Restart: A Night on the Data Link

2026–2027

Statement revision: 29 October 2026

Contents

0 How to read this statement 2

Prologue 4

Q1 Normalize fixed-point telemetry (8 raw points) 5

Q2 Clean and deduplicate damage logs (10 raw points) 7

Q3 Detect anomalous windows (10 raw points) 9

Q4 Execute commands transactionally (12 raw points) 11

Q5 Compute grid neighborhood risk (12 raw points) 13

Q6 Select a priority target (14 raw points) 15

Q7 Simulate heat and ammunition (16 raw points) 17

Q8 Build a telemetry processing pipeline (18 raw points) 19

B1 Challenge: parse a signed decimal integer (+10 points) 21

B2 Challenge: stable in-place compaction (+10 points) 23

Submission and acceptance 25

0 How to read this statement

This is the formal specification of tasks, interfaces, scoring, and submission. See the student README for setup and commands. Copper-colored stories supply motivation; numbered clauses define acceptance. Examples illustrate typical behavior rather than replacing the contract. Private tests check only published behavior.

This assessment covers fundamental syntax and procedural data processing. Plain structs with data fields are allowed. Do not introduce custom classes, constructors, access-control design, inheritance, polymorphism, or ownership design. Use the supplied standard string/vector/array/stream patterns without studying their internal classes. Function wrappers are provided. B1 references and B2 array pointers are optional bridges.

Read background and task, then interfaces and clauses, and verify examples in order. Examples show direct function calls rather than stdin text. PUBLIC 2/2 means two embedded public groups passed, not a full formal score.

Task overview

ID	Task	Topics	Raw weight
Q1	Normalize fixed-point telemetry	Integer arithmetic, branching, string assembly	8
Q2	Clean and deduplicate damage logs	Stream parsing, iteration, Boolean arrays, plain structs	10
Q3	Detect anomalous windows	Sliding windows, indexing, time complexity	10
Q4	Execute commands transactionally	Parsing, branching, transactional updates, counting	12
Q5	Compute grid neighborhood risk	Matrices, nested loops, boundaries, exceptions	12
Q6	Select a priority target	Filtering, lexicographic priorities, integer width	14
Q7	Simulate heat and ammunition	Loop simulation, update order, resource accounting	16
Q8	Build a telemetry processing pipeline	Pipelines, sorting, deduplication, count invariants	18
B1	Challenge: parse a signed decimal integer	Characters, output references, overflow prevention (optional)	+10
B2	Challenge: stable in-place compaction	Array pointers, in-place processing, stability (optional)	+10

Execution, input, and output

Use C++17 and only its standard library, without hardware or external file access. Implementations communicate through arguments, return values, or object state; do not read stdin or print debugging output. Inputs outside stated domains are not assessed unless behavior is explicitly specified. Published invalid cases, exception types, endpoints, and failure states are contractual.

CPP-Classroom runs one complete .cpp per task, containing fixed types, the implementation region, checks, and main. Edit only between BEGIN IMPLEMENTATION and END IMPLEMENTATION; preserve markers, filenames, and interfaces. Ordinary helpers may be added inside; Assessment 2 may add necessary private members. The formal grader extracts this region and recompiles it with trusted types, independently of the student main.

Tasks do not depend on other answers, earlier tests, or shared mutable globals. Persistent state belongs to its object. Throw only when specified; Boolean failure must not silently become an exception.

Prologue

Prologue

In 2027, AIM is already counting down to its next competition trip. The previous replay freezes on an embarrassing scene: plentiful targets, rapidly scrolling logs, and a sentry that never moves. "Process running" clearly does not mean "robot working."

Python training separated perception, navigation, decisions, and missions. C++ now takes over the data link closer to devices. You do not need to build an entire robot overnight. Instead, address a connected set of engineering problems: replayed events, untrusted readings, half-applied commands, and unexplained firing requests.

You are on data-link duty. Give each input a clear classification, each update an explicit rule, and the replay an auditable summary. Everything runs offline without hardware. Stories provide motivation; numbered clauses define acceptance.

Q1 Normalize fixed-point telemetry *(8 raw points)*

Background

On the night before departure for the 2027 league, the familiar startup sound finally returns. Someone celebrates; someone else points at 103% health and -7% battery. Different modules finish initialization at different times, and stale readings can arrive before configuration. The Python sentry can already send notifications. You are moving its critical data path to C++, beginning with a predictable startup report. Every separator belongs to a protocol consumed by another program. A report that merely looks plausible in a terminal is insufficient.

Your task

Implement the ordinary functions `ratio` and `report`. The first computes an integer health percentage; the second builds a startup record. The caller supplies arguments and reads return values. No interactive input or custom type is required.

Topics: Integer arithmetic, branching, string assembly. File: `src/main/A1_q1.cpp`.

Fixed interface and data

```
long long ratio(long long hp, long long max_hp);
std::string report(int id, long long hp, long long max_hp, int battery);
```

Use the existing `aim::q1` namespace in the skeleton; do not nest it again. Fixed structs or base types shown above are already provided outside the implementation region.

Contract

1. The magnitudes of hp and max_hp are at most 10^{12}. id is in [0,9999]; battery may be any int.
2. For max_hp<=0, ratio returns -1 without division. The HP field of report is INVALID, while battery processing still proceeds.
3. Otherwise clamp hp to [0,max_hp], then take the integer part of hp*100/max_hp. Do not use floating-point arithmetic or rounding.
4. Clamp battery to [0,100] first. Values >=60 give OK, values from 20 through 59 give WARNING, and lower values give LOW.
5. Return exactly ID=<id>;HP=<percentage or INVALID>;BAT=<normalized battery>;STATE=<category>, with no zero padding, spaces, or trailing newline.
6. Do not modify inputs or carry results between calls.

Examples and explanation

```
ratio(2, 3);                // 66
report(7, 2, 3, 20);         // "ID=7;HP=66;BAT=20;STATE=WARNING"
report(0, 4, 0, -7);         // "ID=0;HP=INVALID;BAT=0;STATE=LOW"
```

Two thirds becomes 66 by truncation, and battery exactly 20 belongs to WARNING. With an invalid maximum, preserve the rest of the report: health is INVALID and negative battery becomes zero and LOW.

Q2 Clean and deduplicate damage logs *(10 raw points)*

Background

During a test the operator reports one hit, while the replay counts three. Reconnection has replayed buffered events. The log also contains comments, empty lines, and incomplete records.

An earlier cleaner marked an event as seen before validating it. When a malformed event 12 arrived before the real event 12, the real damage vanished. Your cleaner must validate the whole line before deduplicating it. Identifiers refer to events, and each valid event contributes at most once.

Your task

Implement `clean` to convert raw lines into `Damage` records. The provided struct contains only id and value. Preserve the arrival order of first valid events rather than sorting identifiers.

Topics: Stream parsing, iteration, Boolean arrays, plain structs. File: `src/main/A1_q2.cpp`.

Fixed interface and data

```
struct Damage { int id; int value; };
std::vector<Damage> clean(const std::vector<std::string>& lines);
```

Use the existing `aim::q2` namespace in the skeleton; do not nest it again. Fixed structs or base types shown above are already provided outside the implementation region.

Contract

1. There are at most 20000 lines. Each valid line has exactly two decimal int fields, id damage, separated by stream whitespace; surrounding whitespace is permitted.
2. id is in [0,9999], damage in [1,1000]. Use standard decimal integer stream extraction, including optional plus signs and leading zeros. Integer overflow makes a field invalid.
3. Ignore empty lines, comments, missing fields, extra fields, and non-whitespace suffixes after numbers.
4. Complete syntax and range validation first. An invalid line does not reserve an identifier.
5. Retain {id,damage} only for the first valid occurrence of each id. Later valid duplicates never replace it.
6. Do not modify lines. Empty or wholly invalid input returns an empty sequence; deduplication starts fresh on every call.

Examples and explanation

```
clean({"2 5", "bad", "2 9", "1 3"});  // {{2,5},{1,3}}
clean({"1 -1", "1 4", "1 8"});        // {{1,4}}
clean({"1 2 x", "# 2 3", ""});        // {}
```

The first event 2 keeps damage 5; bad is ignored and the duplicate cannot replace it. A negative damage value does not reserve id 1. Extracting two numbers alone does not establish that the entire line is valid.

Q3 Detect anomalous windows *(10 raw points)*

Background

A full-match average looks healthy, yet the sentry sometimes behaves as if power disappeared. Short bursts are hidden by the average. The electronics team asks for fixed-length rolling-window alarms so the relevant video frames can be inspected.
Negative readings represent deviation from a baseline and remain valid. Match logs are long; summing every window from scratch would make analysis too slow. Produce exact alarms with processing time linear in the number of frames.

Your task

Implement `alarms` to return the zero-based right-end indexes of length-k windows whose sums exceed the threshold. Return frame indexes, not window numbers, left endpoints, or an alarm count.

Topics: Sliding windows, indexing, time complexity. File: `src/main/A1_q3.cpp`.

Fixed interface and data

```
std::vector<int> alarms(const std::vector<int>& values, int k, long long threshold);
```

Use the existing `aim::q3` namespace in the skeleton; do not nest it again. Fixed structs or base types shown above are already provided outside the implementation region.

Contract

1. n is at most 200000; each value is in [-10^9,10^9]. The threshold magnitude is at most 10^{15} and k is an int.
2. For k<1 or k>n, including empty input, return an empty sequence without forming partial windows.
3. Windows are [0,k-1], [1,k], through [n-k,n-1]; overlap is allowed.
4. Alarm only when the sum is strictly greater than threshold. Equality does not alarm.
5. Return right-end indexes in ascending order, at most once per window, and leave input unchanged.
6. Require O(n) time and O(1) auxiliary working space, excluding the returned sequence. Use a sufficiently wide integer for sums.

Examples and explanation

```
alarms({1,2,3,4}, 2, 4);    // {2,3}
alarms({2,2}, 2, 4);        // {}
alarms({-4,-3,-2}, 2, -6);  // {2}
```

The first sums are 3,5,7 at indexes 1,2,3, so indexes 2 and 3 alarm. Equality produces no alarm. In the negative example, -5 exceeds -6; negative sums are not automatically safe.

Q4 Execute commands transactionally *(12 raw points)*

Background

A test script says: add five, set eight, reset, subtract two. The SET line also contains an accidental debug field. An old parser updates the value as soon as it reads SET 8 and discovers the invalid suffix too late.

Use an accumulator to practice transactional commands: validate syntax, arguments, and the proposed final range before committing any change. Rejected commands must also be counted so replay statistics account for every request.

Your task

Implement `execute` and return a `CommandResult` containing the final value and accepted/rejected command counts. The struct carries data only and needs no member functions.

Topics: Parsing, branching, transactional updates, counting. File: `src/main/A1_q4.cpp`.

Fixed interface and data

```
struct CommandResult { long long value; int accepted; int rejected; };
CommandResult execute(const std::vector<std::string>& lines);
```

Use the existing `aim::q4` namespace in the skeleton; do not nest it again. Fixed structs or base types shown above are already provided outside the implementation region.

Contract

1. Start at zero and process at most 10000 lines. Each line is one command; an empty line is rejected.
2. Accept only case-sensitive ADD x, SET x, and RESET, allowing surrounding stream whitespace. ADD and SET require exactly one argument; RESET takes none.
3. Parse decimal x with standard stream extraction and require $[-10^6,10^6]$. Reject extra or missing fields, decimal suffixes, unknown operations, and overflow.
4. The candidate result is old+x for ADD, x for SET, and zero for RESET. It must also lie in $[-10^6,10^6]$.
5. Commit the value and increment accepted only after complete validation. Otherwise increment rejected and preserve value.
6. Empty input returns {0,0,0}. accepted+rejected equals the number of lines, and calls are independent.

Examples and explanation

```
execute({"ADD 5","SET 8 x","RESET","ADD -2"}); // {-2,3,1}
execute({"SET 1000000","ADD 1"});            // {1000000,1,1}
```

The first states are 5,5,0,-2: the malformed SET changes nothing. In the second example, argument 1 is valid but its proposed result is out of range, so the previous value remains 1000000.

Q5 Compute grid neighborhood risk *(12 raw points)*

Background

The sentry keeps driving close to pillars. Before building a full navigation cost map, the team wants a simple risk display: each traversable cell counts adjacent obstacles.
One implementation counts diagonals; another treats the edge as a wall. Both pictures look plausible but mean different things downstream. Follow the exact neighborhood and validation contract. Generate a risk map without finding paths or silently repairing input.

Your task

Implement `risk` to turn a character map into an integer matrix of the same dimensions. Both input and output use [row][column] indexing.

Topics: Matrices, nested loops, boundaries, exceptions. File: `src/main/A1_q5.cpp`.

Fixed interface and data

```
std::vector<std::vector<int>> risk(const std::vector<std::string>& grid);
```

Use the existing `aim::q5` namespace in the skeleton; do not nest it again. Fixed structs or base types shown above are already provided outside the implementation region.

Contract

1. The map has at most 300 rows and 300 columns. An empty row sequence is valid and returns an empty matrix. A nonempty map must have at least one column and equal row lengths.
2. Only . for free space and # for obstacles are allowed. Zero columns, unequal rows, or any other character throw std::invalid_argument.
3. Validate the complete map before computing risk; invalid characters in later rows still matter.
4. Obstacle cells produce -1. Free cells count obstacles immediately above, below, left, and right, giving 0..4.
5. Diagonals are excluded and out-of-bounds positions are not obstacles. Edges add no automatic risk.
6. Leave input unchanged and never return a partial map. Require O(rows*columns) time for valid input.

Examples and explanation

```
risk({".#", ".."});       // {{1,-1},{0,1}}
risk({"."});               // {{0}}
risk({".#.","#.#",".#."}); // center cell: 4
risk({"..", "."});         // throws std::invalid_argument
```

Only the top-right cell is blocked in the first map. Its left and lower neighbors have risk one; bottom-left has zero. A single free cell has risk zero. Unequal rows must be rejected instead of padded or truncated.

Q6 Select a priority target *(14 raw points)*

Background

The camera detects armor plates reliably, but the turret oscillates between opponents. Vision prefers the closest target; decision logic prefers low health. The team adopts a deterministic replay rule: filter invalid candidates, then compare health, distance, and identifier.

This is a training protocol rather than a complete competition tactic. The same set of candidates must choose the same identifier even if its input order changes. Ties cannot depend on luck.

Your task

Implement `select_target` to return one Target id. The plain records contain coordinates relative to the robot origin. Range uses Euclidean distance, not Manhattan distance.

Topics: Filtering, lexicographic priorities, integer width. File: `src/main/A1_q6.cpp`.

Fixed interface and data

```
struct Target { int id; int x; int y; int hp; bool visible; };
int select_target(const std::vector<Target>& targets, int max_range);
```

Use the existing `aim::q6` namespace in the skeleton; do not nest it again. Fixed structs or base types shown above are already provided outside the implementation region.

Contract

1. At most 100000 targets have unique nonnegative ids, coordinate magnitudes <=10^6, hp in [-100,100], and a Boolean visible field.
2. Return -1 immediately for negative max_range; nonnegative ranges are at most 10^6. Empty input also returns -1.
3. Eligible targets are visible, have hp>0, and satisfy x*x+y*y<=max_range*max_range, including the boundary.
4. Choose lower hp first; compare squared distance only when hp ties, then lower id only when both tie.
5. Return -1 if none qualify; otherwise return the id rather than its array index. Input order is not a tie-breaker.
6. Require O(n) time without modifying or sorting input. Compute squares and sums with a sufficiently wide integer type.

Examples and explanation

```
select_target({{7,3,4,20,true},{2,0,5,20,true}}, 5); // 2
select_target({{7,3,4,1,true},{2,0,0,2,true}}, 5);   // 7
select_target({{1,0,0,0,true}}, 1);                 // -1
```

Both first-example targets have squared distance 25 and equal health, so id 2 wins. In the second, lower health outranks shorter distance. A zero-health target is ineligible even when located at the origin.

Q7 Simulate heat and ammunition *(16 raw points)*

Background

"I pressed fire" describes a request, not a guaranteed shot. Ammunition may be empty or heat may be too high. Even the order within a frame matters: cooling before checking can allow a shot that checking first would reject.

Implement the exact discrete simulation specified here. These are coursework rules and must not be replaced by another competition heat model. Final counters and remaining resources must agree with one another.

Your task

Implement `simulate` and return a FireResult: actual shots, rejected firing requests, and final ammunition and heat after every frame has been processed.

Topics: Loop simulation, update order, resource accounting. File: `src/main/A1_q7.cpp`.

Fixed interface and data

```
struct FireResult { int shots; int rejected; int ammo; int heat; };
FireResult simulate(int ammo, int heat, int cool, int shot_heat, const std::vector<int>&
    requests);
```

Use the existing `aim::q7` namespace in the skeleton; do not nest it again. Fixed structs or base types shown above are already provided outside the implementation region.

Contract

1. ammo, heat, cool, and shot_heat are in [0,100]. There are at most 10000 binary requests.
2. Every frame begins with heat=max(0,heat-cool).
3. A zero request only cools. It consumes no ammunition and changes neither shots nor rejected.
4. A one request fires only if ammo>0 and cooled heat+shot_heat<=100. Equality at 100 is allowed.
5. A shot consumes one round, adds shot_heat, and increments shots. Failure only increments rejected: no ammunition or shot heat is consumed.
6. Start shots and rejected at zero. Empty requests cause no cooling and preserve initial resources. Calls are independent.

Examples and explanation

```
simulate(2,90,10,20,{1,1,1}); // {2,1,0,100}
simulate(3,100,20,10,{0});    // {0,0,3,80}
simulate(1,100,0,0,{1});      // {1,0,0,100}
```

In the first example frame 1 cools 90 to 80 and fires to 100. Frame 2 cools to 90, but adding 20 would exceed 100, so it rejects. Frame 3 cools to 80 and fires to 100. The result is two shots, one rejection, zero ammunition, and heat 100.

Q8 Build a telemetry processing pipeline *(18 raw points)*

Background

The final replay runs, but its counts do not reconcile: one line was classified as both malformed and duplicate by separate modules. You now define a clear pipeline output that accounts for every input line and supplies a sorted robot-id list.
Here identifiers represent robots within one summary, unlike event ids in Q2. Later valid packets never replace the first valid value. Every line belongs to exactly one category, invalid data cannot reserve an id, and no state may leak from an earlier replay.

Your task

Implement the ordinary function summarize, combining parsing, classification, deduplication, health normalization, and id ordering. Summary contains mutually exclusive valid/duplicate/invalid counts, total normalized health, and sorted ids.

Topics: Pipelines, sorting, deduplication, count invariants. File: `src/main/A1_q8.cpp`.

Fixed interface and data

```
struct Summary { int valid; int duplicate; int invalid; long long total_hp; std::vector<int
    > ids; };
Summary summarize(const std::vector<std::string>& lines);
```

Use the existing `aim::q8` namespace in the skeleton; do not nest it again. Fixed structs or base types shown above are already provided outside the implementation region.

Contract

1. At most 20000 lines each contain exactly two decimal int fields id hp. Surrounding stream whitespace is allowed; extra fields and non-whitespace suffixes are forbidden.
2. id is in [0,9999] and hp in [-1000,1000], parsed with standard stream extraction. Syntax errors, integer overflow, and range violations count as invalid and reserve no id.
3. Only after full validation check duplicates. A previously accepted id increments duplicate and changes no other output.
4. A first valid record increments valid, clamps hp to [0,100] for the long long total_hp, and retains its id.
5. Finally sort ids ascending, rather than preserving Q2 arrival order. Each id appears once.
6. Empty input gives zero counts, zero total, and empty ids. valid+duplicate+invalid equals line count, and valid equals ids.size().

7. Leave input unchanged and do not depend on Q2 or any other task source. This task compiles and scores independently.

Examples and explanation

```
Summary s = summarize({"2 200","1 -5","2 10","bad"});
// s.valid=2, s.duplicate=1, s.invalid=1
// s.total_hp=100, s.ids={1,2}
```

The first hp 200 is legal input and contributes 100 after clamping. hp -5 is also legal and contributes zero. The later id 2 is duplicate and cannot replace 100. bad is invalid. The four lines partition into 2+1+1, and ids become 1,2.

B1 Challenge: parse a signed decimal integer *(+10 points)*

Background

A single numeric conversion can ruin a replay: a large positive value overflows and becomes a negative control value. The team asks for a signed parser that detects overflow before unsafe arithmetic.

Conversion libraries are useful in real programs, but this optional challenge excludes them so you handle signs, digits, asymmetric bounds, and failure behavior yourself. No required task depends on this extension.

Your task

Implement parse_i32. The Boolean indicates success, and the caller-provided output variable may be changed only on success. The supplied reference parameter is the location where a successful result is returned.

Topics: Characters, output references, overflow prevention (optional). File: `src/main/A1_b1.cpp`.

Fixed interface and data

```
bool parse_i32(const std::string& text, int& output);
```

Use the existing `aim::b1` namespace in the skeleton; do not nest it again. Fixed structs or base types shown above are already provided outside the implementation region.

Contract

1. Text length is at most 1000. Valid syntax is one optional + or - followed by at least one ASCII digit 0..9.
2. Allow leading zeros, +0, and -0. Reject all whitespace, decimal points, repeated signs, non-ASCII digits, and trailing characters. Empty text and a sign alone are invalid.
3. The result must fit [-2147483648,2147483647], inclusive; the negative and positive limits are asymmetric.
4. On success assign output and return true. On any failure return false and preserve its original value.
5. Do not use stoi/stol/stoll, string streams, or C numeric-conversion functions. Scan characters and reject before overflowing arithmetic.
6. Require O(text length) time, O(1) auxiliary space, and no custom class.

Examples and explanation

```
int value = 9;
parse_i32("-2147483648", value); // true; value=-2147483648
parse_i32("2147483648", value);  // false; value remains -2147483648
parse_i32("-0000", value);       // true; value=0
```

The minimum signed 32-bit integer is valid. One above the maximum positive value fails while preserving the previous result. Leading zeros are accepted, and negative zero becomes zero.

B2 Challenge: stable in-place compaction *(+10 points)*

Background

A sensor buffer contains outliers. The communication code wants valid readings compacted at its front, with a new usable length. Sorting would destroy time order, and making a second buffer would consume extra memory.
You receive an existing integer array and its length. Rearrange it in place without replacing or freeing it. Retained values keep their sampling order. This optional pointer-and-array exercise introduces no ownership or object-oriented design.

Your task

Implement compact to retain values in [low,high], move them stably to the front, and return their count. Only the prefix before the returned length belongs to the result.

Topics: Array pointers, in-place processing, stability (optional). File: `src/main/A1_b2.cpp`.

Fixed interface and data

```
std::size_t compact(int* data, std::size_t n, int low, int high);
```

Use the existing `aim::b2` namespace in the skeleton; do not nest it again. Fixed structs or base types shown above are already provided outside the implementation region.

Contract

1. n<=200000. For n=0, data may be nullptr and must not be dereferenced. Otherwise it points to n valid int elements.
2. Retain low<=data[i]<=high inclusively, including duplicates.
3. Stability preserves relative order among retained elements; do not sort.
4. After returning k, data[0..k-1] is the result; the remaining tail is unspecified.
5. For low>high, return zero without modifying any element. No matches also returns zero.
6. Require O(n) time and O(1) extra space. No auxiliary container or new array, and do not free the caller's array.

Examples and explanation

```
int data[] = {-1,2,0,9};
std::size_t k = compact(data,4,0,2); // k=2; prefix {2,0}
compact(nullptr,0,0,1);              // 0
```

The retained values are 2 then 0, not sorted as 0 then 2. The result length is two and the tail is unspecified. Empty input returns zero without reading data[0].

Submission and acceptance

Development and submission

Keep task filenames under src/main. A successful CPP-Classroom run still requires active submission. Repository users submit the URL and final main commit; platform users submit their record and requested source export. The instructor announces the deadline.

Edit only implementation regions and the personal notes at the end of README; also commit automatically generated .agent-sessions/bundles/ archives. Put experimental tests in a new local_tests directory. Preserve public tests, fixed interfaces, markers, tools, and workflows. Keep reviewable incremental records by task or fix. Before submission, place your implementations in a fresh template and rerun checks.

Document each task's approach, boundaries, and complexity, at least three additional boundary tests, and one actual defect, diagnosis, and fix. AI assistance is allowed: disclose its scope, how you checked its output, and reviewable process records. If unused, explicitly declare this and supply development records. You must be able to explain your interfaces and failure paths.

AI assistance and Agent session records

AI assistance is allowed and encouraged when used responsibly. Its use alone does not reduce functional credit: you must verify the code and explain interfaces, state changes, and failure paths. The existing 20-point transparency block reviews disclosure, archive coverage, authenticity, and verification. If no AI was used, explicitly state this and retain development records; do not fabricate session archives.

As in the Python Assessment, use Agent Session Commit (AgentLedger), pinned to 0.1.5. Local archiving needs Git and Python 3.10+; running the C++ programs still needs only C++17. Configure each personal student repository, not the private instructor directory or an unrelated project. CPP-Classroom users also synchronize the actual submitted source into their personal Git repository and submit its incremental history and records.

Install in a persistent Python environment, then run the installer from the student repository root:

```
python -m pip install --index-url https://pypi.org/simple 'agent-session-commit[pre-commit
    ]==0.1.5'
agent-session-commit install --pre-commit
agent-session-commit status
```

Select the Agent actually used and confirm its session source. Settings are local to each clone: configure each Assessment and new clone separately, and rerun the installer after upgrading. The supplied .pre-commit-config.yaml pins the archive backend to v0.1.5. Keep the Python environment available; moving or removing it requires reinstalling hooks.

After a successful normal git commit, the post-commit hook archives new session content under .agent-sessions/bundles/ and amends the same commit. Unchanged sessions produce no new bundle or extra archive commit. Amend changes the SHA, so use git rev-parse HEAD for the final revision. A healthy status alone does not establish complete recording: after a real change and commit, inspect the final commit for its corresponding archive.

```
git rev-parse HEAD
git show --stat HEAD -- .agent-sessions/
```

Rerun setup when changing Agent or source. Browser chats, unsupported IDEs, and remote environments are not automatically guaranteed coverage: export actual conversations in a supported format and select Custom when needed. Raw exports may use the ignored .agent-sessions/source/ directory; bundles must be committed. Do not ignore the whole .agent-sessions/ directory. In README, disclose tools, assistance scope, archive location, how results were checked, and any missing records with an explanation and supporting material.

Archives are ordinary, unencrypted Git content without automatic redaction; public pushes make them readable. Select this coursework's source only and exclude credentials and unrelated private sessions. Declare no AI use honestly, explain incomplete records, and do not invent or substitute conversations.

The course cross-checks archives, repository markers, and Git history. Transparent AI use is not penalized. A marker alone never replaces review of development records or determines functional or final grades.

Scoring

Q1–Q8 have raw weights 8,10,10,12,12,14,16,18, totaling 100 and scaled by 0.6 to 60 automatic points. Each fixed test group earns partial credit; a seeded group passes only when all its assertions pass. B1 and B2 add ten each without scaling, for at most twenty bonus points.

Manual assessment contributes 40: process transparency 20 (complete disclosure 5, reviewable development/assistance records 10, verification explanation 5); code quality and explanation 20 (naming and organization 5, implementation/complexity compliance 5, boundary tests and defect analysis 5, design explanation 5). Students not using AI are assessed by equivalent development-record quality. Core maximum is 100 and total maximum including challenges is 120.

Passing public tests does not guarantee private success. Missing files, TODOs, incompatible interfaces, compilation errors, unexpected exceptions, timeouts, fabricated output, and early exits before test completion do not pass. Failure in one task does not block others. Manual fields remain blank and total remains null until instructor assessment; automatic points alone are not the final grade.

Resources and review

The formal grader allows 45 wall-clock seconds per compilation and five per test group, with a 512 MiB execution address-space limit on Linux. Complexity and implementation restrictions also receive source review; a single fast run does not prove compliance. Classroom limits can differ; formal results use the instructor's consistent environment.

Report specification conflicts with a minimal reproduction, actual result, and relevant clause. Preserve tests and do not guess unpublished requirements.